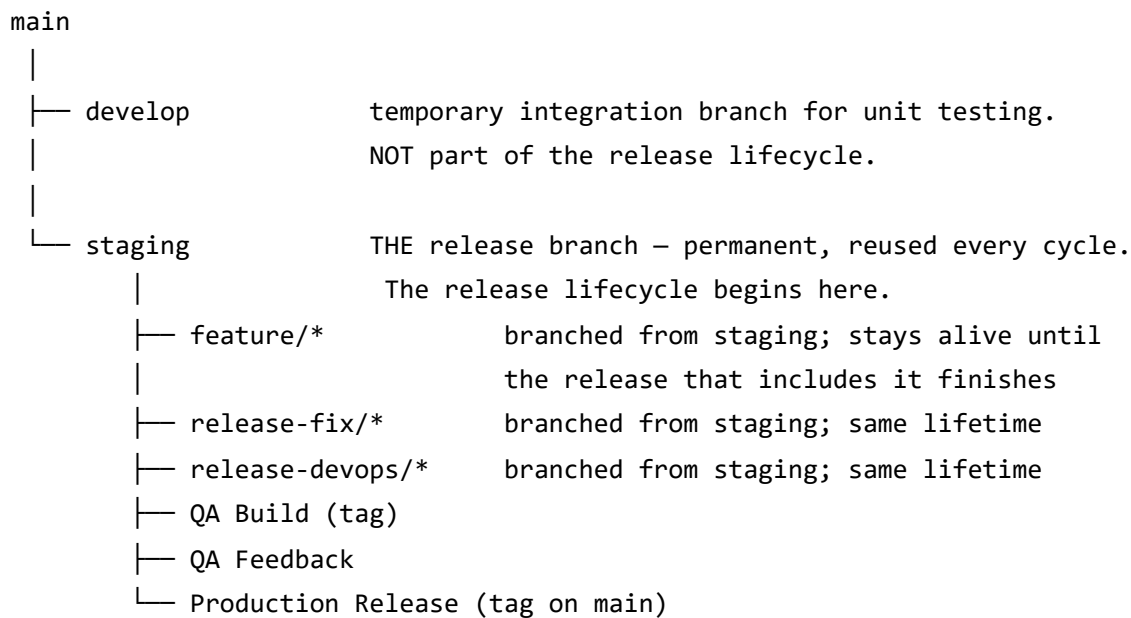


Release Management

This is the part of Git Flow Plus that isn't standard Git Flow. Read this if you're going to run a release, not just contribute code.

The branch model



`staging` is not a temporary branch cut per release and thrown away. It's created once by `git flow init` and lives forever, the same way `main` and `develop` do. Each release cycle (5.2 , then 5.3 , ...) reuses it in turn: `release start` resets its metadata for a new cycle, `release finish` clears that metadata back out when the cycle ends.

`develop` is deliberately outside all of this — it's not where features branch from, and `release finish` never merges into it (see [Feature Management & Release Planning](#) below). It exists purely as a place teams can integrate and unit-test work outside Git Flow Plus's view; the tool doesn't orchestrate anything on it.

The version scheme

`Sprint.Release.Fix.DevOps.QAIteration` — e.g. `5.2.4.1.3` :

Field	Meaning
5	Sprint number
2	Feature counter — how many features have been merged into this release cycle
4	Release fixes included in the current QA build
1	DevOps changes included in the current QA build
3	QA build number

This is not Semantic Versioning. It exists to answer one question: *what, exactly, is in this QA build.*

The second field is a **live counter**, not a fixed identifier: `release start 5.2` sets it to `2` (parsed from the release name), and it

increments by one every time `git flow release feature add` merges a *new* feature into staging (re-running `add` to pull in a developer's QA follow-up commits does **not** increment it again — see [What Release Planning does now](#)). So `5.2` can

become `5.4.0.0.1` after two features land, well before any release fix or QA build has happened. This is independent of the release's own *name* — `"5.2"`, chosen at `release start` — which never changes for the life of the cycle: it's what `release finish 5.2` matches against, what the archive file is named, and what the production tag (`v5.2`) is named. Every tag message spells out both explicitly (`Release: 5.2 / Version: 5.4.1.0.2`), so there's never any ambiguity about which is which.

The rule that matters: merging is not releasing

Merging a release-fix or release-devops branch into staging never changes the version. You can merge ten release-fix branches in a row and the version won't move. This is deliberate: it decouples "a fix landed" from "a fix shipped in a QA build," so:

- Developers can merge fixes continuously without needing to coordinate a version bump for each one.
- The Release Manager decides *when* a batch of fixes becomes an official, tagged, deployable QA build — a conscious act, not a side effect of someone else's merge.
- The version number always corresponds to something a QA engineer can actually pull and test, not an arbitrary mid-merge state.

Concretely:

```
git flow release start 5.2          # staging: 5.2.0.0.1 (QA Build 1, tagged v5.2.0.0.1)
```

```
git flow releasefix finish BUG-101  # staging still 5.2.0.0.1
```

```
git flow releasefix finish BUG-102  # staging still 5.2.0.0.1
```

```
git flow releasefix finish BUG-103  # staging still 5.2.0.0.1
```

```
git flow releasefix finish BUG-104  # staging still 5.2.0.0.1
```

```
git flow release build              # NOW: 5.2.4.0.2 (4 new fixes, QA Build 2, tagged v5.2.4.0.2)
```

```
git flow releasefix finish BUG-201
```

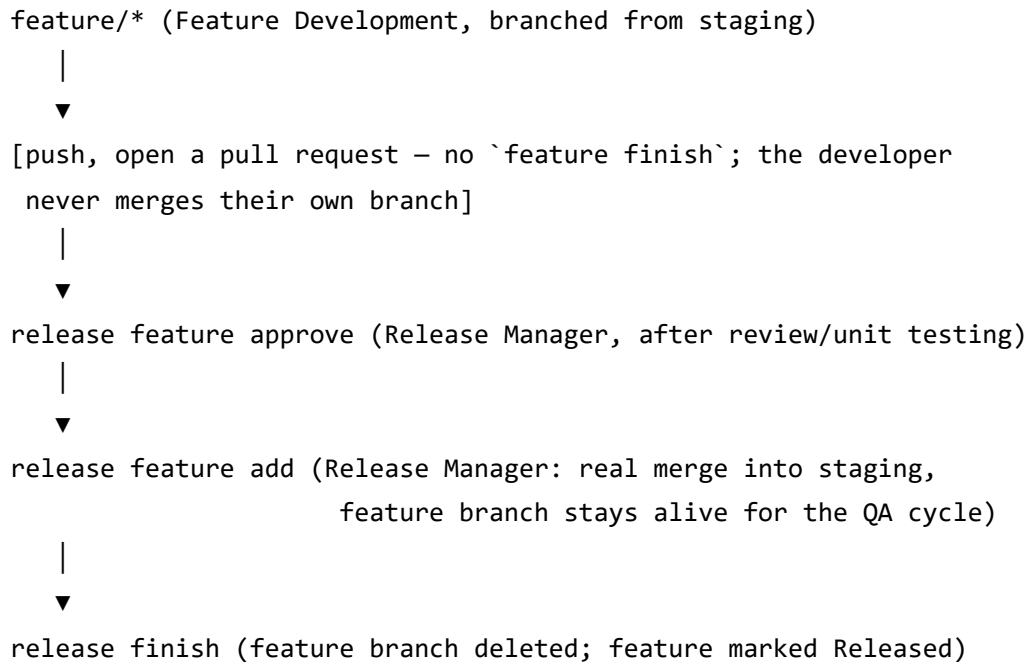
```
git flow releasefix finish BUG-202  # staging still 5.2.4.0.2
```

```
git flow release build              # 5.2.6.0.3 (2 more fixes, QA Build 3, tagged v5.2.6.0.3)
```

`releasefix finish` / `devops finish` only ever do two things: merge the branch into staging and record the change as **pending** in `release.json`. The branch is deliberately **not** deleted — like feature branches, it stays alive through the rest of the QA cycle in case a follow-up commit is needed, and is only deleted in bulk by `release finish` once the release completes (see [Release Completion](#)). `release build` is the only command that reads the pending lists, folds their counts into the version (`Fixes += len(pending fixes)` , `DevOps += len(pending devops)` , `QA += 1`), moves them from pending to included, and appends a build-history entry. Nothing else touches `version.json`'s `Fixes/DevOps/QA` fields — the feature counter (`Release`) is the one exception, advanced separately by `release feature add` (see below).

Feature Management & Release Planning

Git Flow Plus is not just a Git branching tool — it's a release management platform, and a release is a *curated collection of approved features*, merged in one at a time by the Release Manager, not everything that happens to accumulate on some shared branch. The full SDLC it models:



develop doesn't appear in this diagram at all — see [The branch model](#).

The Feature Registry

Every feature branch ever started gets one permanent entry in the **Feature Registry** (`.gitflowplus/features.json`), independent of any single release cycle — unlike `release.json` , it's never reset:

```

{
  "features": [
    {
      "id": "LOGIN",
      "branch": "feature/LOGIN",
      "mergeCommit": "a1b2c3d...",
      "state": "Released",
      "release": "5.3"
    }
  ]
}

```

state tracks the feature's position in its lifecycle:

State	Set by	Meaning
Created	git flow feature start	Branch exists, registered.
InDevelopment	<i>(not automated)</i>	Reserved for manual/future use — Git Flow Plus has no visibility into individual commits.
AwaitingReview	<i>(not automated)</i>	Reserved for manual/future use — pull-request review happens on your Git host, not in this tool.
Approved	git flow release feature approve	Ready for Release Planning.
IncludedInRelease	git flow release feature add	Merged into staging, in the <i>active</i> cycle.
Released	git flow release finish	Shipped — permanent, release field now set.
Archived	<i>(not automated)</i>	Reserved for a possible future cleanup pass.

States only move forward. `git flow feature start` is the only registration point — there's no `feature finish` to hook into, since a developer never finishes their own feature (see below).

release feature commands

Command	What it does
<code>git flow release feature approve <id></code>	Marks a feature <code>Approved</code> , ready for Release Planning.
<code>git flow release feature list</code>	Lists every feature that's reached at least <code>Approved</code> .
<code>git flow release feature add <id></code>	Release Planning decision, and a real merge: validates the branch exists, merges <code>feature/<id></code> into staging, advances the version's feature counter, and marks the feature <code>IncludedInRelease</code> . See below for what re-running it does.
<code>git flow release feature defer <id></code>	Explicitly holds an approved feature back for a future release. Bookkeeping only — no merge.
<code>git flow release feature status</code>	Shows <code>Approved</code> / <code>Included</code> / <code>Deferred</code> / <code>Pending</code> for the active release.

There is no `release feature remove` . Once `add` has performed a real merge, "removing" a feature would mean reverting that merge — an operation with no safe, generally-correct definition, so Git Flow Plus doesn't attempt it. `defer` is the tool for holding a feature back, and it only works *before* `add` has merged anything.

Git Flow Plus never assumes every approved feature belongs to the next release. A feature stays `Pending` — approved, but with no release decision made yet — until a Release Manager explicitly runs `add` or `defer` . Given LOGIN, PROFILE, PAYMENT, and REPORTS are all approved, a Release Manager might decide only LOGIN, PROFILE, and PAYMENT ship in 5.3 , deferring REPORTS to 5.4 :

```
git flow release feature approve LOGIN
git flow release feature approve PROFILE
git flow release feature approve PAYMENT
git flow release feature approve REPORTS
```

```
git flow release feature add LOGIN      # merges feature/LOGIN into staging; Release: 5.3 -> 5.4
git flow release feature add PROFILE    # merges feature/PROFILE; Release: 5.4 -> 5.5
git flow release feature add PAYMENT    # merges feature/PAYMENT; Release: 5.5 -> 5.6
git flow release feature defer REPORTS  # bookkeeping only, no merge
```

Pending is never hand-maintained — it's always recomputed as *approved minus included-or-shipped minus deferred*, so it can't drift out of sync with the registry.

What Release Planning does now

Unlike bookkeeping-only planning tools, `release feature add` performs the actual merge — this is the point where a feature's code genuinely becomes part of the release, not just a metadata label. That has one direct consequence the design leans into rather than works around: **the feature branch is never deleted by `add`**. It has to stay alive, because QA can still report an issue against that feature, and the fix for it has to go somewhere — the same branch, not a new one, so the history stays attached to the one feature.

When that happens, the developer pushes a follow-up commit to the still-alive `feature/<id>` branch. Git Flow Plus does not watch for that or sync it automatically — the Release Manager notices (via their normal QA process) and simply **re-runs the same command**:

```
git flow release feature add LOGIN      # first time: merges, advances the feature counter
# ...QA finds an issue, developer pushes a fix to feature/LOGIN...
git flow release feature add LOGIN      # re-run: merges the new commit, feature counter untouched
```

A repeat call for a feature already IncludedInRelease *in the active cycle* merges again (a harmless no-op if there's genuinely nothing new) without incrementing the version's feature counter a second time — a feature is counted once, no matter how many times its branch gets synced. Calling `add` for a feature that already shipped in a *finished* release

(Released / Archived) is rejected — that's a different feature-set entirely and there's nothing to sync.

Release Completion

Only `git flow release finish` deletes anything. Until then, every `feature/*` branch that was added, every `release-fix/*`, and every `release-devops/*` branch merged this cycle all stay alive, precisely so follow-up commits have somewhere to land. `release finish`:

1. Creates the production tag and merges `staging` into `main` (as before).
2. Archives the release manifest.
3. Marks every included feature `Released` in the registry.
4. Deletes every feature branch that was included in this release, every `release-fix/*` branch, and every `release-devops/*` branch.
5. Resets the release state on `staging` for the next cycle.

Branch deletion (step 4) is best-effort: if a branch somehow can't be deleted cleanly (e.g. it has commits that were never merged — which shouldn't happen if `add` / `finish` were used as intended, but Git won't silently discard history either way), Git Flow Plus logs a warning and moves on rather than failing an otherwise-successful release.

The release manifest

`release.json`'s `features` block mirrors this directly, and a `featureHistory` audit trail records every `add` action, including resyncs:


```

{
  "release": "5.3",
  "branch": "staging",
  "currentVersion": "5.6.2.1.4",
  "currentQABuild": 4,
  "features": {
    "included": ["LOGIN", "PROFILE", "PAYMENT"],
    "deferred": ["REPORTS"],
    "pending": ["DASHBOARD"]
  },
  "releaseFixes": { "included": ["BUG-101"], "pending": [] },
  "devops": { "included": [], "pending": ["redis-cache"] },
  "history": [ ... ],
  "featureHistory": [
    {
      "id": "LOGIN",
      "branch": "feature/LOGIN",
      "mergeCommit": "a1b2c3d...",
      "version": "5.4.0.0.1",
      "addedAt": "2026-07-27T10:00:00Z"
    }
  ]
}

```

release build does **not** fold "pending" features into "included" the way it does for release fixes and DevOps changes — feature inclusion happens immediately, at add time, not on a batch cadence. release build does still snapshot whichever features are currently included into that build's history entry and QA tag, so every QA build's Feature List is always accurate.

Tagging

Every QA build gets an annotated tag — this is not optional.

Deployments key off these tags, so a build without one isn't deployable.

- `git flow release start` tags the initial version (QA Build 1) — `v5.2.0.0.1`.
- `git flow release build` tags every subsequent build — `v5.2.4.0.2`, `v5.2.6.0.3`, etc.

- `git flow release finish` creates a **separate** production tag, named after the release itself — `v5.2` — not the full version string. That name is already taken by the last QA build's tag (pointing at a different commit on staging), and Git tag names must be globally unique, so the production tag can't reuse it. `v5.2` marks the one production release of 5.2; its message still records the exact version and commit that shipped.

Every tag's message is structured and complete, so you never need to dig through commit history to know what a tag represents:

```
Release: 5.2
Version: 5.2.4.0.2
QA Build: 2
Release Date: 2026-07-24T10:42:34Z
Release Manager: Jane Doe <jane@example.com>
```

Features:

```
LOGIN
PROFILE
```

Release Fixes:

```
BUG-101
BUG-102
BUG-103
BUG-104
```

DevOps:

```
None
```

Branch: `staging`

Commit: `a1b2c3d4e5f6...`

(The production tag's message is the same shape, with `Branch: main` and no `QA Build` line — a production release isn't itself a QA iteration.)

Lifecycle hooks (CI/CD integration)

Git Flow Plus stays CI/CD-agnostic: it never talks to GitHub Actions, GitLab CI, Azure DevOps, Jenkins, or Bitbucket Pipelines directly. Two

ways to trigger a pipeline off a release event:

1. **Trigger off the tag push itself.** Since every QA build and every production release is a real annotated tag, any CI/CD system configured to watch tag patterns (e.g. `v*`) will fire automatically once you `git push --tags`. This needs no configuration in Git Flow Plus at all.
2. **Use a lifecycle hook script**, for anything that needs to happen locally, before a push, or independent of tag-watching (e.g. calling a webhook directly, writing to a local queue).

To use a hook, drop an executable script at:

```
.gitflowplus/hooks/post-qa-tag[.sh|.ps1|.bat|.cmd]
.gitflowplus/hooks/post-production-tag[.sh|.ps1|.bat|.cmd]
```

Pick the extension for your platform: `.sh` (run via `sh`), `.ps1` (run via `powershell -File`), `.bat` / `.cmd` (run via `cmd /C`), or no extension at all (run directly — relies on a shebang line and the executable bit on Unix). Git Flow Plus tries them in that order and runs the first one it finds; if none exist, nothing happens (hooks are entirely opt-in and never cause a command to fail just because a script is missing).

Commit hook scripts to `staging` — that's the branch `release start` / `release build` run from, and where `post-qa-tag` fires. The `post-production-tag` hook fires after `release finish`'s merge sequence, which now ends on `main` (develop is untouched — see [The branch model](#)), so a hook needed there should be committed on `main` too (or, simplest: commit hook scripts once on `main` before `staging` diverges — `git flow init` seeds `config.json` onto `main` / `staging` / `develop` this same way, and hook scripts can ride along).

Each hook receives these environment variables:

Variable	post-qa-tag	post-production-tag
GITFLOWPLUS_EVENT	qa-build	production-release
GITFLOWPLUS_RELEASE	✓ (e.g. <code>5.2</code>)	✓
GITFLOWPLUS_VERSION	✓ (e.g. <code>5.2.4.0.2</code>)	✓ (full version that shipped)

Variable	post-qa-tag	post-production-tag
GITFLOWPLUS_TAG	✓ (e.g. v5.2.4.0.2)	✓ (e.g. v5.2)
GITFLOWPLUS_QA_BUILD	✓	—
GITFLOWPLUS_COMMIT	—	✓
GITFLOWPLUS_BRANCH	✓ (staging)	✓ (main)

A hook script that exits non-zero is logged as a warning but does **not** fail the release command or undo the tag — the tag and version bookkeeping are already committed by the time hooks run, and a broken deployment trigger shouldn't be able to corrupt release state.

The full command sequence

```
git flow init                                # main, staging, develop

git flow release start 5.2                   # staging: version.json = 5.2.0.0.1
                                           # tags v5.2.0.0.1 (QA Build 1)

git flow feature start LOGIN                # feature/LOGIN, from staging
# ...commit, push, open a PR – no `feature finish`...
git flow release feature approve LOGIN
git flow release feature add LOGIN          # merges feature/LOGIN into staging; version -> 5.3.0.0.1
                                           # feature/LOGIN stays alive

git flow releasefix start BUG-101           # release-fix/BUG-101, from staging
# ...fix, commit...
git flow releasefix finish BUG-101          # merge into staging; BUG-101 now PENDING (branch stays alive)

git flow devops start redis-cache           # release-devops/redis-cache, from staging
# ...change, commit...
git flow devops finish redis-cache          # merge into staging; redis-cache now PENDING (branch stays alive)

git flow release status                    # see what's pending vs. included

git flow release build                      # fold pending -> included, version -> 5.3.1.1.2
                                           # tags v5.3.1.1.2 (QA Build 2)
                                           # fires post-qa-tag hook

# ...QA validates the build; suppose QA flags an issue in LOGIN...
# developer pushes a fix to feature/LOGIN, then:
git flow release feature add LOGIN          # re-run: merges the fix, version untouched this time

git flow release build                      # another QA build if needed, e.g. version -> 5.3.1.1.3

# ...QA signs off...

git flow release finish 5.2                 # requires: nothing pending (run build first)
                                           # archives release.json -> .gitflowplus/archive/5.2.json
                                           # removes live release.json/version.json (reset for next release)
                                           # merges staging -> main (develop untouched)
                                           # tags v5.2 (production) at the exact commit that merged :
                                           # marks LOGIN Released; deletes feature/LOGIN,
                                           #   release-fix/BUG-101, release-devops/redis-cache
```

```
# fires post-production-tag hook
```

```
git flow release start 5.3 # staging reused, fresh cycle: version.json = 5.3.0.0.1
```

release finish refuses to run while anything is pending
(ErrPendingChangesNotBuilt) — nothing should ship without ever having moved the version and been through a tagged QA build.

release status

Shows everything about the release active on staging right now:

```
Release:          5.2
Branch:           staging
Version:          5.2.4.0.2
QA Build:         2
Included Release Fixes:  BUG-101, BUG-102, BUG-103, BUG-104
Pending Release Fixes:  (none)
Included DevOps:      (none)
Pending DevOps:       (none)
Included Features:    LOGIN, PROFILE, PAYMENT
Deferred Features:    REPORTS
Pending Features:     DASHBOARD
Open Release-Fix Branches: (none)
Open DevOps Branches:  (none)
```

"Active" means "does staging 's current working tree have a release.json " — there's no separate global pointer to track. Run it from any other branch and you'll see "No active release on staging." (release status always reports on staging specifically, regardless of which branch you're currently on, matching where release state actually lives.)

Known gaps

- **One release at a time.** release start refuses to run if a release is already active on staging (ErrReleaseAlreadyActive). There's no support for two releases in flight simultaneously.
- **No way to un-merge a feature.** There is deliberately no release feature remove — once add has performed a real merge, reverting it

safely and generally isn't something Git Flow Plus attempts. `defer` only works before the first `add`.

- **No automatic detection of follow-up commits.** The Release Manager has to notice (via their normal QA process) that a feature branch got a new commit and re-run `release feature add` — Git Flow Plus doesn't poll or watch branches for changes.

See [Roadmap.md](#) for what's planned to close these.